

```

# Sugestões de Melhoria - Front-end Django MQTT
## 1. Adicionar Autenticação via JWT
### Modificar `settings.py`
```python
REST_FRAMEWORK = {
SIMPLE_JWT = {
Criar arquivo `monitoring/auth.py`
```python
class ProtectedAPIView:
def token_pair(request):
### Modificar `views.py`
```python
Manter api_state para frontend (sem auth)
Proteger api_command com autenticação
try:
Validar token
... resto do código

2. Parametrizar Threshold de Temperatura
Modificar `settings.py`
```python
### Modificar `state.py`
```python
def ingest_message(topic: str, payload: str) -> None:
if topic.endswith("temperatura"):
Usar threshold configurável

3. Persistência de Histórico em Banco de Dados
Criar modelo `monitoring/models.py`
```python
class MQTTMessage(models.Model):
topic = models.CharField(max_length=255)
class Meta:
def __str__(self):
class TemperatureReading(models.Model):
class Meta:
### Modificar `state.py`
```python
def ingest_message(topic: str, payload: str) -> None:
if topic.endswith("temperatura"):
Persistir em BD
_record_history_entry()

4. Implementar WebSocket para Tempo Real
Instalar dependências
```bash
### Criar `monitoring/consumers.py`
```python
class DashboardConsumer(AsyncWebsocketConsumer):
async def disconnect(self, close_code):
async def monitor_state(self):
async def receive(self, text_data):
Configurar `settings.py`

```

```

```python
ASGI_APPLICATION = "fleet_front.asgi.application"
CHANNEL_LAYERS = {
### Criar `monitoring/routing.py`
```python
websocket_urlpatterns = [
Modificar `fleet_front/asgi.py`
```python
os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'fleet_front.settings')
application = ProtocolTypeRouter({
---
## 5. Adicionar Logging Estruturado
### Criar `monitoring/logger.py`
```python
class MQTTJSONFormatter(logging.Formatter):
def format(self, record):
if record.exc_info:
return json.dumps(log_data)
Configurar em settings.py

6. Rate Limiting para API
Modificar `views.py`
```python
def rate_limit(max_calls, time_window=60):
call_count = cache.get(cache_key, 0)
if call_count >= max_calls:
cache.set(cache_key, call_count + 1, time_window)
def get_client_ip(request):
@require_POST
---
## 7. Testes Unitários
### Criar `monitoring/tests/test_mqtt.py`
```python
class MQTTStateTestCase(TestCase):
def setUp(self):
def test_temperature_ingestion(self):
snapshot = get_snapshot()
def test_temperature_alert(self):
snapshot = get_snapshot()
def test_invalid_temperature(self):
snapshot = get_snapshot()
def test_status_ingestion(self):
snapshot = get_snapshot()
def test_command_validation(self):
success, msg = publish_command("ON")
def test_thread_safety(self):
threads = [threading.Thread(target=publish_messages) for _ in range(3)]
snapshot = get_snapshot()
Executar testes
```bash
---
## 8. Arquivo `.env` Recomendado
```bash

```

# MQTT Configuração  
# MQTT Tópicos  
# MQTT Configuração Avançada  
# Banco de Dados  
# Segurança  
# Logging

---

## Priorização de Implementação  
### Fase 1 (Imediato - 1 semana)  
### Fase 2 (Curto prazo - 2-3 semanas)  
### Fase 3 (Médio prazo - 1 mês)  
### Fase 4 (Longo prazo - 2+ meses)

---

\*\*Estimativa de esforço total\*\*: 80-100 horas de desenvolvimento